

docker

Prácticas — parte 2: imágenes y Dockerfile

Antes de empezar

- Puedes usar el entorno en la nube propuesto (tipo Play with Docker) o tu propio Docker Desktop/VM/WSL2.
- Necesitarás una cuenta gratuita en [Docker Hub](#) para el último ejercicio.
- Ve resolviendo los ejercicios en orden, cada uno se apoya en el anterior.
- Los **retos** (al final) no traen los pasos, solo el objetivo: decides tú los comandos.

Ejercicios guiados

Ejercicio 1 — Tu primera imagen

1. Crea una carpeta `practica-docker` con un fichero `index.html` con contenido a tu elección.
2. Crea un `Dockerfile` que, a partir de `nginx:alpine`, copie `index.html` a `/usr/share/nginx/html/`.
3. Construye la imagen con el nombre `mi-web:1.0`.
4. Comprueba con `docker images` que aparece.

Resultado esperado: `docker images` muestra `mi-web` con tag `1.0`.

Ejercicio 2 — Ejecútala y publícala

1. Ejecuta un contenedor a partir de `mi-web:1.0` publicando el puerto 80 del contenedor en el 8080 de tu máquina.
2. Abre `http://localhost:8080` (o la URL equivalente de tu entorno) y comprueba que ves tu página.
3. Para el contenedor y bórralo.

Resultado esperado: ves tu `index.html` en el navegador a través del puerto 8080.

Ejercicio 3 — Una app con dependencias

1. Crea una carpeta nueva `practica-app` con un pequeño script (Node, Python o el lenguaje que prefieras) que imprima algo o levante un mini-servidor.
2. Si tu lenguaje usa gestor de dependencias (`package.json` , `requirements.txt` ...), créalo con al menos una dependencia.
3. Escribe un `Dockerfile` que: copie primero el fichero de dependencias, instale dependencias, copie después el resto del código, y defina `CMD` para ejecutar tu script.
4. Construye la imagen como `mi-app:1.0` .

Resultado esperado: `docker build` termina sin errores y `docker images` muestra `mi-app:1.0` .

Ejercicio 4 — Aprovecha la caché

1. Ejecuta de nuevo `docker build` sobre `mi-app:1.0` sin cambiar nada. Fíjate en los mensajes "Using cache".
2. Modifica solo el código (no las dependencias) y reconstruye. ¿Qué capas se reconstruyen y cuáles no?
3. Modifica ahora el fichero de dependencias y reconstruye. ¿Qué cambia esta vez?

Resultado esperado: sabes explicar, con tus palabras, por qué cambiar el código no obliga a reinstalar dependencias si el Dockerfile está bien ordenado.

Ejercicio 5 — Variables de entorno y volumen

1. Añade a tu `Dockerfile` una instrucción `ENV` para un valor que tu app pueda leer (p. ej. un puerto o un mensaje).
2. Reconstruye la imagen y ejecútala sobrescribiendo esa variable con `-e` en `docker run`.
3. Ejecuta el contenedor montando un volumen (`-v`) que apunte a una carpeta de datos (puede estar vacía).
4. Comprueba que un fichero creado dentro de esa carpeta, dentro del contenedor, aparece también en el host.

Resultado esperado: el valor de la variable cambia según lo que pases en `-e`, y los ficheros del volumen persisten en el host.

Ejercicio 6 — Publica tu imagen en Docker Hub

1. Haz `docker login` con tu cuenta de Docker Hub.
2. Retaguea `mi-app:1.0` como `tu_usuario/mi-app:1.0`.
3. Súbela con `docker push`.
4. Borra la imagen local y descárgala de nuevo con `docker pull tu_usuario/mi-app:1.0`.

Resultado esperado: tu imagen aparece en tu perfil de Docker Hub y puedes volver a descargarla.

Retos

Aquí no hay pasos: piensa qué instrucciones y comandos necesitas.

Reto 1 — Dockerfile mal escrito, a propósito

Te dan (o te escribes tú mismo) un `Dockerfile` que copia todo el código **antes** de instalar las dependencias.

Objetivo: reordénalo para aprovechar la caché correctamente y demuestra, con dos builds (antes/después de un cambio de código), que ahora es más rápido.

Reto 2 — Reduce el tamaño de tu imagen

Objetivo: consigue que tu imagen `mi-app` ocupe menos espacio en disco (usa `docker images` para comparar), usando una imagen base más ligera y/o eliminando herramientas que solo hacían falta durante el build.

Reto 3 — Dos contenedores que se hablan

Objetivo: crea una red Docker personalizada, levanta dos contenedores conectados a ella (por ejemplo, tu app y una base de datos como `mysql` o `postgres`) y demuestra que uno puede comunicarse con el otro **por nombre**, sin publicar puertos al host.

Reto 4 — El puerto ocupado

Objetivo: provoca a propósito un error de "puerto ya en uso" ejecutando dos contenedores publicando el mismo puerto del host, interpreta el mensaje de error y resuélvelo sin parar ningún contenedor (usando otro puerto de host).

Autoevaluación

¿Te ves capaz de...?

- [] Escribir un `Dockerfile` básico con `FROM`, `WORKDIR`, `COPY`, `RUN` y `CMD`.
- [] Explicar la diferencia entre `CMD` y `ENTRYPOINT`.
- [] Explicar por qué el orden de las instrucciones afecta a la caché de build.
- [] Construir una imagen con `docker build -t nombre:tag .`.
- [] Publicar y descargar una imagen propia en Docker Hub.
- [] Usar un volumen (`-v`) para persistir datos de un contenedor.